

# Project Status Report: BragBoard - Week 2 Completion

**E.Saipavan**

**Email:** sesikela@gitam.in

**Github:** <https://github.com/saipavanesikela/Bragboard>

**Date:** October 9, 2025

**Status:** All Week 2 tasks are complete.

## Overview of Week 2 Tasks

The primary goal for Week 2 was to build the core user interface and add department-level logic to the backend. The specific tasks were:

- Build a basic layout with Tailwind CSS.
- Display the user dashboard after login.
- Set up department-wise scoping in the backend.

## 2. What We Have Done (Implementation Details)

### Task 1: Build a Basic Layout

A comprehensive and modular layout structure was created for the application.

**Initial Layout:** A primary Layout.jsx component was built, which included a shared <Header> and <Footer>.

**Dashboard Layout:** A more specific and feature-rich dashboard layout was implemented, separating the interface into three main components:

**Sidebar-navigation.jsx:** For consistent navigation.

**TopNavbar.jsx:** For profile and notification access.

**MainContentArea.jsx:** To display dynamic content like the shout-out feed and user stats.

**Styling:** All styles were initially built with Tailwind CSS and later refactored into separate, standard CSS files (SharedStyles.css, DashboardPage.css) for better organization and maintainability.

### Task 2: Display the User Dashboard After Login

The dashboard is now the main landing page for authenticated users.

**Dashboard Creation:** A DashboardPage.jsx was created to serve as the central hub, containing a welcome message, quick stats cards, a shout-out feed, and a leaderboard.

**Protected Routing:** A ProtectedRoute.jsx component was implemented. This component checks if a user is logged in using the AuthContext. If the user is not authenticated, it automatically redirects them to the login page, securing the dashboard.

**Login Redirect:** The LoginPage.jsx was updated to navigate users to /dashboard upon a successful login.

### **Task 3: Set up Department-Wise Scoping in Backend**

The backend is now equipped to handle department-specific logic.

**Data Collection:** The frontend registration page (RegisterPage.jsx) was updated to include a "Department" dropdown menu, ensuring this data is captured during user creation.

**Backend Logic:** The backend registration endpoint was modified to validate that an email address is unique within the same department, adding a layer of data integrity. The API for fetching shout-outs was also updated to allow filtering by department.

### **3. Challenges Faced and Solutions**

#### **Challenge: Persistent frontend environment instability.**

**Problem:** We encountered repeated npm errors, such as could not determine executable to run, and connection refused errors (net::ERR\_CONNECTION\_REFUSED) even after multiple clean installs. This was due to an unstable Node.js environment.

**Solution:** The issue was definitively resolved by switching the package manager from npm to Yarn. This provided a more stable and reliable environment for installing dependencies and running scripts.

#### **Challenge: Backend connectivity and CORS errors.**

**Problem:** The frontend was unable to communicate with the backend ("Failed to fetch") because of the browser's Cross-Origin Resource Sharing (CORS) policy.

**Solution:** We implemented the CORSMiddleware in the main FastAPI application (main.py), explicitly granting permission to the frontend's origin (http://localhost:5173) to make requests.

#### **Challenge: Multiple frontend rendering issues (blank pages).**

**Problem:** The application frequently failed to render, showing only a blank white page. This was caused by a series of JavaScript SyntaxError issues.

**Solution:** We debugged this by systematically checking the browser's developer console. The errors were traced to components that were missing the export default statement. By adding this line to HomePage.jsx, Layout.jsx, DashboardPage.jsx, and other components, we resolved the import/export conflicts and fixed the rendering.

#### 4. Conclusion and What We Have Learned

Week 2 was highly productive. We not only completed all planned frontend and backend tasks but also overcame significant technical hurdles related to the development environment.

Key Learnings:

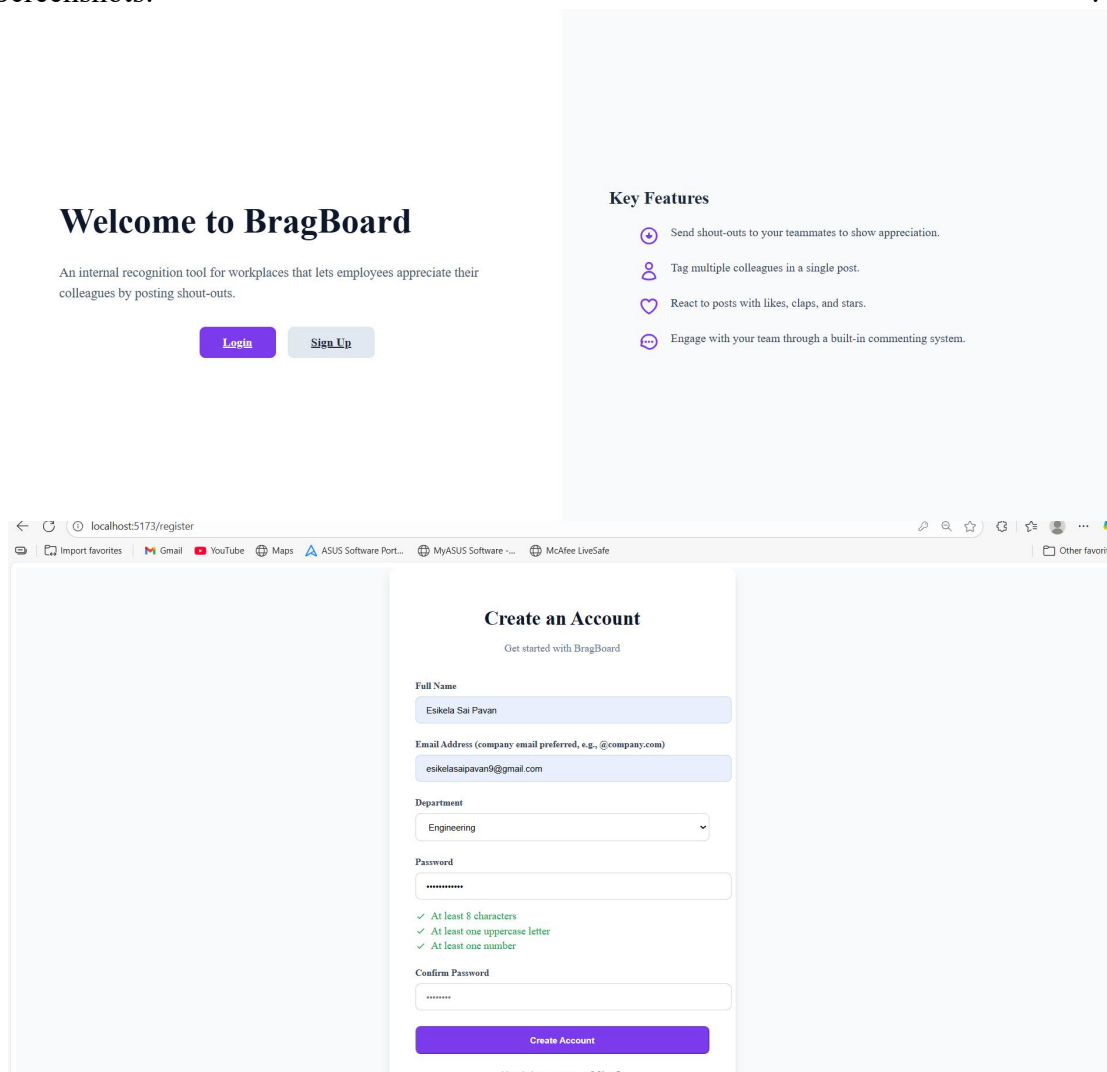
**The Importance of a Stable Environment:** The challenges with npm highlighted how crucial a stable and correctly configured development environment (Node.js version, package manager) is to a project's success.

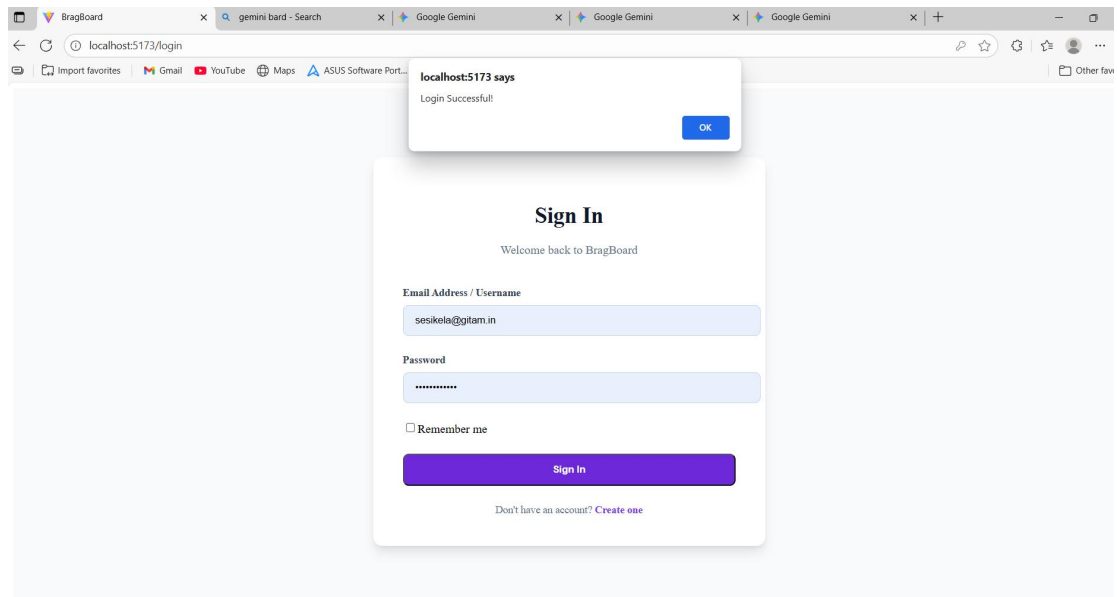
**Systematic Debugging:** The blank page errors reinforced the importance of using the browser's developer console as the primary tool for debugging frontend issues. By isolating components and analyzing console output, we could quickly pinpoint and resolve errors.

**Backend Configuration is Critical:** The CORS and database connection issues were a reminder that backend services require explicit configuration to communicate with other applications and services securely.

**React Fundamentals:** We reinforced our understanding of core React concepts, particularly the module system (imports/exports) and the difference between default and named exports.

Screenshots:





BragBoard

AR

